Since you're preparing deeply for **Java 8 Streams (interview + real-time coding)**, here is a **carefully curated high-impact list**:

□ PART 1: 50 MOST IMPORTANT STREAMS INTERVIEW QUESTIONS (WITH ANSWERS)

✅ SECTION A — Core Concepts (1–10)

1. What is Stream API?

Stream API (introduced in Java 8) is used to process collections using functional-style operations like filter, map, reduce without modifying the original data.

---

2. Difference between Collection and Stream?

Collection	Stream
Stores data	Processes data
Eager	Lazy
Can iterate multiple times	Can iterate only once
External iteration	Internal iteration

---

3. What is lazy evaluation?

Intermediate operations execute only when a terminal operation is called.

```
list.stream()
   .filter(x -> x > 5)   // not executed yet
   .count();             // execution starts here
```

---

4. What are intermediate operations?

They return a Stream.

Examples: filter(), map(), sorted(), distinct()

---

5. What are terminal operations?

They produce a result or side-effect.

Examples: collect(), count(), forEach(), reduce()

---

6. What is map()?

Transforms elements.

```
List<String> names = List.of("A", "B");
List<Integer> lengths = names.stream()
    .map(String::length)
    .collect(Collectors.toList());
```

---

7. Difference between map() and flatMap()?

map()	flatMap()
One-to-one mapping	One-to-many flattening
Returns Stream<Stream>	Returns Stream

---

8. What is reduce()?

Combines elements into a single result.

```
int sum = list.stream()
        .reduce(0, Integer::sum);
```

---

9. Can a Stream be reused?

No. Once consumed, it throws IllegalStateException.

---

10. What is Optional?

A container object that may or may not contain a value.

✅ SECTION B — Collectors & Grouping (11–20)

11. What is collect()?

Used to accumulate stream elements into List, Set, Map etc.

---

12. Example of groupingBy()

```
Map<String, List<Employee>> deptMap =
employees.stream()
        .collect(Collectors.groupingBy(Employee::getDepartment));
```

---

13. Difference: groupingBy vs partitioningBy

groupingBy	partitioningBy
Any key	Boolean key
Multiple groups	Only 2 groups

---

14. How to handle duplicate keys in toMap()?

```
Collectors.toMap(
  Employee::getId,
  Employee::getName,
  (oldValue, newValue) -> oldValue
)
```

---

15. How to count elements group-wise?

```
Collectors.groupingBy(
   Employee::getDept,
   Collectors.counting()
);
```

---

16. How to find max salary per department?

```
Collectors.groupingBy(
   Employee::getDept,
   Collectors.maxBy(Comparator.comparing(Employee::getSalary))
);
```

---

17. What is downstream collector?

Collector applied after grouping.

---

18. How to join strings?
Collectors.joining(", ")

---

19. What is collectingAndThen()?

Post-process collected result.

---

20. What is teeing()?

Java 12 feature to combine two collectors.

✅ SECTION C — Parallel Streams (21–30)

21. What is parallelStream()?

Processes data in multiple threads using ForkJoinPool.

---

22. When NOT to use parallel stream?

- Small data
- I/O bound tasks
- Stateful lambdas

---

23. What is ForkJoinPool?

Framework for parallel execution using work-stealing algorithm.

---

24. What is Spliterator?

Object that splits data for parallel processing.

---

25. Why reduce requires associative function?

Because partial results combine in parallel.

---

26. Difference: findFirst vs findAny

findFirst() preserves order.
findAny() may return any element (faster in parallel).

---

27. What is non-interference?

Stream should not modify source data.

---

28. What is stateless lambda?

Lambda that does not depend on mutable external state.

---

29. Are streams thread-safe?

No. But parallel stream operations are internally managed safely.

---

30. What is combiner?

Used in parallel stream to merge partial results.

✅ SECTION D — Functional Interfaces (31–40)

31. What functional interfaces used in Streams?

- Predicate
- Function
- Consumer
- Supplier
- BinaryOperator

---

32. Predicate chaining?

p1.and(p2)
p1.or(p2)
p1.negate()

---

33. Function composition?

f1.andThen(f2)
f1.compose(f2)

---

34. What is target typing?

Compiler determines lambda type based on context.

---

35. What is effectively final?

Variable whose value does not change after initialization.

---

36. Lambda vs Anonymous class?

Lambda	Anonymous
Less boilerplate	More verbose
No this reference	Has own this

37–40.

37. What is method reference?
38. What is primitive stream?
39. What is boxed()?
40. What is summaryStatistics()?

✅ SECTION E — Advanced & Internal (41–50)

41. How lazy evaluation improves performance?
42. What is infinite stream?
43. Stream.iterate() vs generate()?
44. How short-circuit works?
45. What is pipeline?

46. What is stateful operation?
47. Can streams modify original collection?
48. Why streams are not data structure?
49. How to debug stream?
50. Limitations of Streams API?

Now we'll go **deep + interview-focused** on these core Stream concepts with clear Java examples and internal explanation.

✅ 1. What is Method Reference?

🧠 Definition

A **method reference** is a shorthand syntax of lambda expression when a method already exists.

Instead of:

```
n -> System.out.println(n)
```

We write:

```
System.out::println
```

🧠 Types of Method References

1 Static Method Reference

```
import java.util.*;

public class Test {
   public static void main(String[] args) {
      List<String> list = Arrays.asList("java", "stream");

      list.stream()
         .map(String::toUpperCase)
         .forEach(System.out::println);
   }
}
```

---

2 Instance Method Reference

```
String str = "hello";
Runnable r = str::toUpperCase;
```

---

3 Constructor Reference

```
Supplier<List<String>> supplier = ArrayList::new;
```

```
List<String> list = supplier.get();
```

✔ Interview Insight

Method reference improves readability and reduces boilerplate.

✅ 2. What is Primitive Stream?

📘 Definition

Specialized stream for primitive types:

- IntStream
- LongStream
- DoubleStream

Avoids boxing overhead.

Example

```
import java.util.stream.*;

public class Test {
   public static void main(String[] args) {
      IntStream.range(1, 5)
            .forEach(System.out::println);
   }
}
```

✔ Why Important?

Stream<Integer> → Boxing/unboxing overhead
IntStream → More efficient

✅ 3. What is boxed()?

📘 Definition

Converts primitive stream to wrapper stream.

```
IntStream.range(1,5)
        .boxed()
        .forEach(System.out::println);
```

Converts:

IntStream → Stream<Integer>

When Needed?

When using collectors like:

```
List<Integer> list = IntStream.range(1,5)
                        .boxed()
                        .collect(Collectors.toList());
```

✅ 4. What is summaryStatistics()?

Used to get multiple statistics in one pass.

Example:

```
import java.util.*;
import java.util.stream.*;

public class Test {
   public static void main(String[] args) {
      List<Integer> list = Arrays.asList(1,2,3,4,5);

      IntSummaryStatistics stats = list.stream()
                                 .mapToInt(Integer::intValue)
                                 .summaryStatistics();

      System.out.println("Sum: " + stats.getSum());
      System.out.println("Average: " + stats.getAverage());
      System.out.println("Max: " + stats.getMax());
      System.out.println("Min: " + stats.getMin());
      System.out.println("Count: " + stats.getCount());
   }
}
```

✔ Why Efficient?

Single traversal → O(n)

✅ 5. How Lazy Evaluation Improves Performance?

Intermediate operations are NOT executed until terminal operation is called.

Example:

```
import java.util.*;

public class Test {
   public static void main(String[] args) {
      List<Integer> list = Arrays.asList(1,2,3,4,5);

      list.stream()
         .filter(n -> {
            System.out.println("Filtering " + n);
            return n > 3;
         })
         .findFirst();
   }
}
```

Output:

```
Filtering 1
Filtering 2
Filtering 3
Filtering 4
```

Stops once match found.

✔ Only required elements processed

✔ Improves performance

✅ 6. What is Infinite Stream?

A stream that produces infinite elements.

Example:

```
Stream.iterate(1, n -> n + 1)
    .limit(5)
    .forEach(System.out::println);
```

Without limit() → infinite loop.

✅ 7. Stream.iterate() vs Stream.generate()

iterate()	generate()
Uses seed + function	Uses Supplier
Deterministic sequence	Random / constant generation

---

iterate()

```
Stream.iterate(1, n -> n + 2)
    .limit(5)
    .forEach(System.out::println);
```

Output:

1 3 5 7 9

---

generate()

```
Stream.generate(Math::random)
    .limit(3)
    .forEach(System.out::println);
```

✅ 8. How Short-Circuit Works?

Stops execution early when result found.

Example:

```
List<Integer> list = Arrays.asList(1,2,3,4,5);

boolean result = list.stream()
            .anyMatch(n -> {
               System.out.println(n);
               return n > 3;
            });
```

Stops once 4 found.

✔ Improves efficiency

✔ Works with findFirst, anyMatch, limit

✅ 9. What is Pipeline?

Stream pipeline consists of:

Source → Intermediate → Terminal

Example:

```
List<Integer> result = list.stream()      // Source
   .filter(n -> n > 2)                    // Intermediate
   .map(n -> n * 2)                       // Intermediate
   .collect(Collectors.toList());         // Terminal
```

Pipeline executes only after terminal operation.

✅ 10. What is Stateful Operation?

Operation that depends on entire stream data.

Examples:

- sorted()
- distinct()
- limit()

Example:

```
list.stream()
   .sorted()
   .forEach(System.out::println);
```

Needs full data before sorting.

✅ 11. Can Streams Modify Original Collection?

❌ No (should not).

Wrong:

```
list.stream()
   .forEach(n -> list.add(n));
```

Throws ConcurrentModificationException.

Streams follow **non-interference principle**.

✅ 12. Why Streams Are Not Data Structure?

Because:

- Do not store elements
- Cannot access by index
- Cannot reuse
- Process data only

Stream is a **view**, not storage.

✅ 13. How to Debug Stream?

Use peek().

```
list.stream()
   .filter(n -> n > 2)
   .peek(n -> System.out.println("After filter: " + n))
   .map(n -> n * 2)
   .collect(Collectors.toList());
```

Or break into steps.

✅ 14. Limitations of Streams API

✔ Hard to debug complex pipelines

✔ No checked exception support

✔ Cannot reuse stream

✔ Not good for heavy I/O

✔ Parallel streams share common ForkJoinPool

✔ Stateful lambdas cause issues

□ Final Interview Summary

✔ Method reference = cleaner lambda

✔ Primitive streams improve performance

✔ boxed() converts primitive to wrapper

✔ summaryStatistics() single-pass stats

✔ Lazy evaluation = efficiency

✔ Infinite streams need short-circuit

✔ Streams are pipelines, not containers

✔ Stateful ops need buffering

✔ Streams should not modify source

If you want next:

✅ 40 FAANG-Level Stream Questions
✅ Deep Reduce & Collector Internals
✅ Parallel Stream Performance Deep Dive
✅ Custom Spliterator Implementation

□ PART 2: 25 REAL INTERVIEW CODING PROGRAMS (WITH FULL JAVA CODE)

Below are the most frequently asked ones.

---

1 Find Duplicate Elements

```
List<Integer> list = Arrays.asList(1,2,3,2,4,1);

Set<Integer> duplicates = list.stream()
    .filter(n -> Collections.frequency(list, n) > 1)
    .collect(Collectors.toSet());

System.out.println(duplicates);
```

---

2 First Non-Repeated Character

```
String str = "swiss";

Character result = str.chars()
    .mapToObj(c -> (char) c)
    .filter(c -> str.indexOf(c) == str.lastIndexOf(c))
    .findFirst()
```

```
    .orElse(null);

System.out.println(result);
```

---

3 Frequency of Each Character

```
Map<Character, Long> map =
str.chars()
  .mapToObj(c -> (char) c)
  .collect(Collectors.groupingBy(c -> c, Collectors.counting()));
```

---

4 Second Highest Number

```
Integer secondHighest =
list.stream()
   .sorted(Comparator.reverseOrder())
   .skip(1)
   .findFirst()
   .orElse(null);
```

---

5 Find Highest Salary Employee

```
Employee emp = employees.stream()
      .max(Comparator.comparing(Employee::getSalary))
      .orElse(null);
```

---

6 Count Even and Odd

```
Map<Boolean, List<Integer>> result =
list.stream()
   .collect(Collectors.partitioningBy(n -> n % 2 == 0));
```

---

7 Flatten List<List>

```
List<Integer> flat =
nestedList.stream()
        .flatMap(List::stream)
        .collect(Collectors.toList());
```

---

8 Sort Map by Values

```
map.entrySet()
  .stream()
  .sorted(Map.Entry.comparingByValue())
  .forEach(System.out::println);
```

---

9 Find Missing Number (1–N)

```
int n = 5;
```

```
int sum = IntStream.rangeClosed(1, n).sum();
int actual = list.stream().mapToInt(Integer::intValue).sum();

System.out.println(sum - actual);
```

Top 3 Highest Numbers

```
list.stream()
   .sorted(Comparator.reverseOrder())
   .limit(3)
   .forEach(System.out::println);
```

Remaining Common Programs (11–25)

11. Merge two lists remove duplicates
12. Group anagrams
13. Reverse each string
14. Convert list to map
15. Longest string
16. Average salary per department
17. Find employees older than 30
18. Sum of all numbers
19. Find palindrome strings
20. Remove null values
21. Sort employees by multiple fields
22. Convert list to comma-separated string
23. Check if list contains duplicates
24. Partition by salary > 50k
25. Convert map to list

(If you want, I can provide all remaining 15 with full code in next message.)

If You Want Next Level:

Since you're already preparing deeply:

- ✅ FAANG-level tricky Stream questions
- ✅ Output-based tricky MCQs
- ✅ Parallel stream internal diagram explanation
- ✅ 50 Stream coding problems with increasing difficulty
- ✅ Stream + Optional advanced interview guide

Great ▯ Since you're deeply working with **Java 8, Collections, Functional Interfaces, and Streams**, here is a **category-wise 120+ Java Streams API interview question bank** — structured the way real interviews ask (basic → intermediate → advanced → scenario based → internal working).

✅ 1. Basics of Streams (15 Questions)

1. What is Stream API in Java 8?
2. Difference between Collection and Stream?
3. How do you create a Stream?
4. What are intermediate and terminal operations?
5. What is lazy evaluation in streams?
6. What is the difference between map() and flatMap()?
7. How does filter() work?
8. What is forEach() vs forEachOrdered()?
9. What is peek() used for?
10. What happens if a stream has no terminal operation?
11. Can a stream be reused?
12. What is the difference between stream() and parallelStream()?
13. How do you convert a Stream to a List?
14. What is Optional in Streams?
15. What is the internal iteration concept?

Since you're preparing deeply for **Java 8 Streams (core + internal understanding)**, here is a **detailed conceptual + practical explanation** of all 15 questions with **clear Java code examples**.

✅ 1. What is Stream API in Java 8?

▯ Definition

Stream API (introduced in Java 8) is used to process collections in a **functional programming style**.

It allows:

- Filtering
- Mapping
- Sorting
- Grouping
- Reducing

Without modifying the original data.

Example

```
import java.util.*;
import java.util.stream.*;

public class StreamExample {
   public static void main(String[] args) {
      List<Integer> list = Arrays.asList(1, 2, 3, 4, 5);

      List<Integer> result = list.stream()
                              .filter(n -> n % 2 == 0)
                              .map(n -> n * 2)
                              .collect(Collectors.toList());

      System.out.println(result); // [4, 8]
   }
}
```

✅ 2. Difference between Collection and Stream?

Feature	Collection	Stream
Purpose	Stores data	Processes data
Iteration	External	Internal
Reusability	Yes	No
Execution	Eager	Lazy
Traversal	Multiple times	Only once

Example Difference

```
List<String> list = Arrays.asList("A", "B", "C");

// Collection
for(String s : list) {
   System.out.println(s);
}

// Stream
list.stream().forEach(System.out::println);
```

✅ 3. How do you create a Stream?

From Collection

```
list.stream();
```

From Array
Arrays.stream(array);

Using Stream.of()
Stream<String> stream = Stream.of("A", "B", "C");

Infinite Stream
Stream<Integer> infinite = Stream.iterate(1, n -> n + 1);

✅ 4. What are intermediate and terminal operations?

Intermediate Operations

- Return Stream
- Lazy
- Can be chained

Examples:

- filter()
- map()
- sorted()
- distinct()

Terminal Operations

- Produce result
- Trigger execution

Examples:

- collect()
- count()
- forEach()
- reduce()

Example

```
list.stream()
   .filter(n -> n > 2)     // intermediate
   .map(n -> n * 2)        // intermediate
   .collect(Collectors.toList());  // terminal
```

✅ 5. What is lazy evaluation in streams?

Stream operations do not execute until a terminal operation is called.

```
list.stream()
   .filter(n -> {
      System.out.println("Filtering " + n);
      return n > 2;
   });  // Nothing prints

list.stream()
   .filter(n -> {
      System.out.println("Filtering " + n);
      return n > 2;
   })
   .count();  // Now executes
```

Lazy evaluation improves performance.

✅ 6. Difference between map() and flatMap()?

map()	flatMap()
One-to-one	One-to-many
Returns Stream<Stream>	Returns Stream
Used for transformation	Used for flattening

---

map() Example

```
List<String> list = Arrays.asList("Java", "Stream");

List<Integer> lengths = list.stream()
                        .map(String::length)
                        .collect(Collectors.toList());
```

---

flatMap() Example

```
List<List<Integer>> nested = Arrays.asList(
      Arrays.asList(1,2),
      Arrays.asList(3,4)
);

List<Integer> flat = nested.stream()
                     .flatMap(List::stream)
                     .collect(Collectors.toList());

System.out.println(flat); // [1,2,3,4]
```

✅ 7. How does filter() work?

It takes a Predicate and keeps elements that return true.

```
List<Integer> result = list.stream()
                .filter(n -> n % 2 == 0)
                .collect(Collectors.toList());
```

Internally:

- Each element passes through predicate
- If true → included
- If false → skipped

✅ 8. forEach() vs forEachOrdered()

forEach()	forEachOrdered()
May not maintain order (parallel)	Maintains order
Faster in parallel	Slower in parallel

Example

```
list.parallelStream()
   .forEach(System.out::println);

list.parallelStream()
   .forEachOrdered(System.out::println);
```

✅ 9. What is peek() used for?

Used for debugging. It is intermediate operation.

```
list.stream()
   .filter(n -> n > 2)
   .peek(n -> System.out.println("Filtered: " + n))
   .map(n -> n * 2)
   .collect(Collectors.toList());
```

⚠ Should not modify state inside peek.

✅ 10. What happens if stream has no terminal operation?

Nothing executes.

```
list.stream()
```

```
    .filter(n -> n > 2); // No output
```

Because of lazy evaluation.

✅ 11. Can a Stream be reused?

No.

```
Stream<Integer> stream = list.stream();
stream.count();
stream.count();  // Throws IllegalStateException
```

Reason:
Stream is consumed after terminal operation.

✅ 12. Difference between stream() and parallelStream()

stream()	parallelStream()
Sequential	Parallel
Single thread	Multiple threads
Predictable order	May break order
Better for small data	Better for large data

Example:

```
list.parallelStream()
    .map(n -> n * 2)
    .collect(Collectors.toList());
```

Uses ForkJoinPool.

✅ 13. How to convert Stream to List?

Java 8

```
List<Integer> result = list.stream()
                    .collect(Collectors.toList());
```

Java 16+

```
List<Integer> result = list.stream().toList();
```

✅ 14. What is Optional in Streams?

Optional prevents NullPointerException.

Used with:

- findFirst()
- max()
- min()

Example:

```
Optional<Integer> max = list.stream()
                    .max(Integer::compare);

max.ifPresent(System.out::println);
```

If no value:

```
Integer value = max.orElse(0);
```

✅ 15. What is internal iteration concept?

External Iteration (Before Java 8)

```
for(Integer n : list) {
   System.out.println(n);
}
```

Developer controls loop.

Internal Iteration (Stream)

```
list.stream().forEach(System.out::println);
```

Stream API controls iteration internally.

Benefits:

- Less code
- Better optimization
- Parallel support

 Summary

Streams:

- Are not data structures
- Are lazy
- Cannot be reused
- Support functional programming
- Allow parallel processing
- Use internal iteration
- Rely on functional interfaces

If you want next:

✅ Deep internal working (Spliterator + Pipeline diagram)
✅ 25 advanced stream coding problems with solutions
✅ Stream vs Loop performance comparison
✅ Stream + Optional advanced usage
✅ FAANG tricky output-based questions

✅ 2. Intermediate Operations (20 Questions)

1. Explain filter()
2. Explain map()
3. Explain flatMap() with example
4. What is distinct()?
5. What is sorted() and how does it work?
6. Difference between sorted() and Comparator
7. What is limit()?
8. What is skip()?
9. What is peek()?
10. How does mapToInt() differ from map()?
11. What are primitive streams?
12. What is boxed()?
13. How to chain multiple intermediate operations?
14. How does stream pipeline work?
15. What is stateless vs stateful intermediate operations?
16. Which operations are stateful?
17. What is flatMapToInt()?
18. How does unordered() work?
19. How does takeWhile() and dropWhile() work (Java 9)?
20. What happens when order is not maintained?

Excellent
Now we are going **deep into intermediate operations** of Java Streams.

I'll explain each concept clearly with:

- ✔ Definition
- ✔ Internal working
- ✔ Java example
- ✔ Interview insight

✅ 1. Explain filter()

Definition

filter() is an **intermediate operation** that selects elements based on a condition (Predicate).

Syntax

```
Stream<T> filter(Predicate<? super T> predicate)
```

Example

```
import java.util.*;
import java.util.stream.*;

public class FilterExample {
   public static void main(String[] args) {
      List<Integer> list = Arrays.asList(1,2,3,4,5,6);

      List<Integer> evens = list.stream()
                        .filter(n -> n % 2 == 0)
                        .collect(Collectors.toList());

      System.out.println(evens); // [2,4,6]
   }
}
```

Internal Working

- Each element passes through Predicate
- If true → forwarded
- If false → dropped

✔ Stateless operation

✔ Lazy execution

✅ 2. Explain map()

▯ Definition

Transforms each element into another form.

Stream<R> map(Function<? super T, ? extends R> mapper)

Example

```
List<String> names = Arrays.asList("Java", "Stream");

List<Integer> lengths = names.stream()
                        .map(String::length)
                        .collect(Collectors.toList());

System.out.println(lengths); // [4,6]
```

✔ One-to-one transformation

✔ Stateless

✅ 3. Explain flatMap() with example

▯ Problem

If we use map() on nested list:

Stream<Stream<Integer>>

We need flattening.

Example

```
List<List<Integer>> nested = Arrays.asList(
    Arrays.asList(1,2),
    Arrays.asList(3,4)
);
```

```
List<Integer> flat = nested.stream()
                .flatMap(List::stream)
                .collect(Collectors.toList());

System.out.println(flat); // [1,2,3,4]
```

✔ Converts Stream<Stream> → Stream

✔ One-to-many transformation

✅ 4. What is distinct()?

Removes duplicate elements.

```
List<Integer> list = Arrays.asList(1,2,2,3,3,4);

List<Integer> unique = list.stream()
                .distinct()
                .collect(Collectors.toList());

System.out.println(unique); // [1,2,3,4]
```

✔ Uses equals() and hashCode()

✔ Stateful operation (needs memory to track seen elements)

✅ 5. What is sorted() and how does it work?

Sorts stream elements.

Natural Order

```
list.stream()
   .sorted()
   .forEach(System.out::println);
```

Custom Comparator

```
list.stream()
   .sorted((a,b) -> b - a)
   .forEach(System.out::println);
```

✔ Stateful operation

✔ Collects elements internally before sorting

✅ 6. Difference between sorted() and Comparator

sorted()	Comparator
Stream method	Comparison logic
Performs sorting	Defines sorting rule

Example:

```
Comparator<Integer> desc = (a,b) -> b - a;
list.stream().sorted(desc);
```

✅ 7. What is limit()?

Limits number of elements.

```
list.stream()
   .limit(3)
   .forEach(System.out::println);
```

✔ Short-circuiting operation

✔ Useful for infinite streams

✅ 8. What is skip()?

Skips first N elements.

```
list.stream()
   .skip(2)
   .forEach(System.out::println);
```

✅ 9. What is peek()?

Used for debugging. Intermediate operation.

```
list.stream()
   .filter(n -> n > 2)
   .peek(n -> System.out.println("After filter: " + n))
   .map(n -> n * 2)
   .collect(Collectors.toList());
```

⚠ Should not modify state.

✅ 10. How does mapToInt() differ from map()?

map()

Returns Stream

mapToInt()

Returns IntStream (primitive)

Example:

```
List<String> list = Arrays.asList("Java","Stream");

int totalLength = list.stream()
                .mapToInt(String::length)
                .sum();

System.out.println(totalLength);
```

✔ Avoids boxing overhead

✔ More performance

✅ 11. What are primitive streams?

Specialized streams for primitives:

- IntStream
- LongStream
- DoubleStream

Example:

```
IntStream.range(1,5).forEach(System.out::println);
```

✅ 12. What is boxed()?

Converts primitive stream to wrapper stream.

```
IntStream.range(1,5)
        .boxed()
        .collect(Collectors.toList());
```

Converts:

IntStream → Stream<Integer>

✅ 13. How to chain multiple intermediate operations?

```
List<String> result = list.stream()
       .filter(s -> s.length() > 3)
       .map(String::toUpperCase)
       .sorted()
       .distinct()
       .collect(Collectors.toList());
```

✔ All intermediate operations are lazy

✔ Executed in single pipeline

✅ 14. How does stream pipeline work?

Pipeline consists of:

1. Source
2. Intermediate operations
3. Terminal operation

Example:

```
list.stream()      // source
   .filter(n -> n > 2)
   .map(n -> n * 2)
   .collect(Collectors.toList());  // terminal
```

Internally:

- Elements processed one by one
- Lazy
- Uses Spliterator

✅ 15. Stateless vs Stateful Intermediate Operations

Stateless

Does not depend on previous elements.

Examples:

- filter()
- map()

Stateful

Depends on other elements.

Examples:

- sorted()
- distinct()

✅ 16. Which operations are stateful?

- sorted()
- distinct()
- limit() (partially)
- skip()

Because they need buffering or previous data.

✅ 17. What is flatMapToInt()?

Used when flattening to primitive int stream.

```
List<List<Integer>> nested = Arrays.asList(
    Arrays.asList(1,2),
    Arrays.asList(3,4)
);

int sum = nested.stream()
        .flatMapToInt(list -> list.stream().mapToInt(Integer::intValue))
        .sum();

System.out.println(sum);
```

✅ 18. How does unordered() work?

Removes encounter order constraint.

```
list.parallelStream()
   .unordered()
   .limit(3)
   .forEach(System.out::println);
```

✔ Improves performance in parallel streams

✅ 19. takeWhile() and dropWhile() (Java 9)

takeWhile()

Takes elements until condition fails.

```
List<Integer> list = Arrays.asList(1,2,3,4,1,2);

list.stream()
   .takeWhile(n -> n < 4)
   .forEach(System.out::println);
// Output: 1 2 3
```

---

dropWhile()

Drops elements until condition fails.

```
list.stream()
   .dropWhile(n -> n < 4)
   .forEach(System.out::println);
// Output: 4 1 2
```

Works only on ordered streams.

✅ 20. What happens when order is not maintained?

In parallel streams:

- Order may not be preserved
- Performance increases
- findAny() becomes faster

Example:

```
list.parallelStream()
   .forEach(System.out::println);  // order not guaranteed
```

If order required:

```
list.parallelStream()
   .forEachOrdered(System.out::println);
```

 Interview Summary

✔ Intermediate operations are lazy

✔ Some are stateless, some stateful

✔ Primitive streams improve performance

✔ flatMap flattens nested streams

✔ Ordering affects parallel performance

✔ Pipeline executes only after terminal operation

If you want next:

✅ Terminal operations deep explanation
✅ Parallel stream internal working
✅ 30 real-time advanced coding problems
✅ Stream performance comparison with loops
✅ FAANG tricky stream questions

✅ 3. Terminal Operations (20 Questions)

1. What is collect()?
2. What is reduce()?
3. Difference between collect() and reduce()?
4. What is count()?
5. What is min() and max()?
6. What is findFirst() vs findAny()?
7. What is allMatch()?
8. What is anyMatch()?
9. What is noneMatch()?
10. What is toArray()?
11. How does forEach() behave in parallel streams?
12. What is short-circuiting operation?

13. How does summaryStatistics() work?
14. What is joining() collector?
15. How to group elements?
16. What is partitioningBy()?
17. How to convert stream into Map?
18. How to handle duplicate keys in toMap()?
19. What is collectingAndThen()?
20. What is averagingInt()?

Excellent
Now we'll cover **Terminal Operations in Java Streams (Detailed + Java Code + Interview Insight)**.

Terminal operations:

- Trigger execution
- Produce result
- Close the stream

✅ 1. What is collect()?

Definition

collect() is used to accumulate stream elements into a **collection or custom result container**.

Syntax

```
<R,A> R collect(Collector<? super T,A,R> collector)
```

Example – Convert to List

```
import java.util.*;
import java.util.stream.*;

public class CollectExample {
   public static void main(String[] args) {
      List<Integer> list = Arrays.asList(1,2,3,4);

      List<Integer> result = list.stream()
                              .filter(n -> n % 2 == 0)
                              .collect(Collectors.toList());

      System.out.println(result); // [2,4]
   }
}
```

🔹 How it works internally (Parallel Concept)

Collector has:

- Supplier (create container)
- Accumulator (add element)
- Combiner (merge partial results)
- Finisher (optional transformation)

✅ 2. What is reduce()?

🔹 Definition

reduce() combines elements into a single result using an associative function.

🔹 Example – Sum

```
List<Integer> list = Arrays.asList(1,2,3,4);

int sum = list.stream()
        .reduce(0, (a,b) -> a + b);

System.out.println(sum); // 10
```

🔹 Three Forms

1. Optional<T> reduce(BinaryOperator)
2. T reduce(identity, BinaryOperator)
3. U reduce(identity, accumulator, combiner)

🔹 Example (Without identity)

```
Optional<Integer> sum = list.stream()
                .reduce((a,b) -> a + b);
```

✅ 3. Difference between collect() and reduce()

reduce()	collect()
Combines values	Collects into container
Returns single value	Returns collection/object

Immutable reduction	Mutable reduction
Simpler use cases	Complex grouping use cases

Example:

```
// reduce
int sum = list.stream().reduce(0, Integer::sum);

// collect
List<Integer> newList = list.stream().collect(Collectors.toList());
```

✅ 4. What is count()?

Counts number of elements.

```
long count = list.stream()
            .filter(n -> n > 2)
            .count();

System.out.println(count);
```

✔ Returns long

✔ Terminal operation

✅ 5. What is min() and max()?

Return smallest/largest element using Comparator.

Example

```
Optional<Integer> min = list.stream()
                .min(Integer::compareTo);

Optional<Integer> max = list.stream()
                .max(Integer::compareTo);

min.ifPresent(System.out::println);
max.ifPresent(System.out::println);
```

✔ Returns Optional

✅ 6. findFirst() vs findAny()

findFirst()	findAny()
Maintains order	May return any
Slower in parallel	Faster in parallel

Example

```
Optional<Integer> first = list.stream()
                    .filter(n -> n > 2)
                    .findFirst();
```

Parallel:

```
list.parallelStream()
   .filter(n -> n > 2)
   .findAny();
```

✅ 7. What is allMatch()?

Checks if all elements satisfy condition.

```
boolean result = list.stream()
              .allMatch(n -> n > 0);

System.out.println(result);
```

✔ Short-circuiting

✅ 8. What is anyMatch()?

Returns true if any element matches.

```
boolean result = list.stream()
              .anyMatch(n -> n > 3);
```

Stops as soon as one match found.

✅ 9. What is noneMatch()?

Returns true if no elements match condition.

```
boolean result = list.stream()
                .noneMatch(n -> n < 0);
```

✅ 10. What is toArray()?

Converts stream to array.

```
Object[] array = list.stream().toArray();
```

Typed version:

```
Integer[] arr = list.stream().toArray(Integer[]::new);
```

✅ 11. How does forEach() behave in parallel streams?

In parallel:

- Order NOT guaranteed

```
list.parallelStream()
   .forEach(System.out::println);
```

If order required:

```
list.parallelStream()
   .forEachOrdered(System.out::println);
```

✅ 12. What is short-circuiting operation?

Stops processing once result is determined.

Examples:

- findFirst()
- findAny()
- anyMatch()
- allMatch()
- noneMatch()
- limit()

Example:

```
list.stream()
```

```
.anyMatch(n -> {
    System.out.println(n);
    return n > 2;
});
```

Stops once condition true.

✅ 13. How does summaryStatistics() work?

Works on primitive streams.

```
IntSummaryStatistics stats = list.stream()
                        .mapToInt(Integer::intValue)
                        .summaryStatistics();

System.out.println(stats.getSum());
System.out.println(stats.getAverage());
System.out.println(stats.getMax());
System.out.println(stats.getMin());
System.out.println(stats.getCount());
```

✔ Returns summary object

✔ Efficient single pass

✅ 14. What is joining() collector?

Joins strings.

```
List<String> names = Arrays.asList("Java","Stream");

String result = names.stream()
                .collect(Collectors.joining(", "));

System.out.println(result); // Java, Stream
```

With prefix & suffix:

```
Collectors.joining(", ", "[", "]");
```

✅ 15. How to group elements?

Using groupingBy().

```
Map<Integer, List<Integer>> grouped =
list.stream()
   .collect(Collectors.groupingBy(n -> n % 2));

System.out.println(grouped);
```

✅ 16. What is partitioningBy()?

Special case of groupingBy for Boolean condition.

```
Map<Boolean, List<Integer>> result =
list.stream()
   .collect(Collectors.partitioningBy(n -> n % 2 == 0));
```

Only 2 groups: true / false

✅ 17. How to convert Stream into Map?

```
Map<Integer, Integer> map =
list.stream()
   .collect(Collectors.toMap(
      n -> n,
      n -> n * n
   ));
```

✅ 18. How to handle duplicate keys in toMap()?

```
Map<Integer, Integer> map =
list.stream()
   .collect(Collectors.toMap(
      n -> n % 2,
      n -> n,
      (oldValue, newValue) -> oldValue
   ));
```

Third argument = merge function

✅ 19. What is collectingAndThen()?

Used to apply finishing transformation after collecting.

```
List<Integer> unmodifiable =
list.stream()
   .collect(Collectors.collectingAndThen(
      Collectors.toList(),
      Collections::unmodifiableList
   ));
```

✅ 20. What is averagingInt()?

Calculates average of int values.

```
double avg = list.stream()
            .collect(Collectors.averagingInt(n -> n));

System.out.println(avg);
```

Equivalent to:

```
list.stream().mapToInt(n -> n).average();
```

□ Final Interview Insights

✔ Terminal operations trigger execution

✔ reduce → immutable aggregation

✔ collect → mutable container aggregation

✔ Short-circuit improves performance

✔ Optional prevents NullPointerException

✔ Parallel streams affect ordering

✔ summaryStatistics is optimized

If you want next:

✅ Custom Collector implementation
✅ Parallel stream internal working diagram
✅ 30 Advanced real interview coding problems
✅ Stream vs Loop performance benchmark
✅ FAANG tricky stream questions

✅ 4. Collectors & Grouping (20 Questions)

1. Explain Collectors.groupingBy()
2. Multi-level grouping example
3. Difference between groupingBy() and partitioningBy()
4. How to count elements in each group?
5. How to sum values group-wise?
6. How to find max salary employee per department?
7. How to flatten nested collections?
8. How to collect into Set?
9. How to collect into LinkedList?
10. How to create custom Collector?
11. What is downstream collector?
12. How does mapping() work?
13. What is reducing() collector?
14. What is teeing() collector (Java 12)?
15. Immutable collectors in Java 10+
16. How to sort map using stream?
17. How to group by multiple fields?
18. How to convert List to Map<Key,List>?
19. What is concurrent collector?
20. How to collect in parallel safely?

Now we'll go deep into **Collectors & Advanced Grouping (Very Important for Interviews)**.

I'll explain each with:

- ✔ Concept
- ✔ Internal behavior
- ✔ Java code
- ✔ Interview insight

✅ 1. Explain Collectors.groupingBy()

▯ Definition

Groups elements based on a classification function.

 Syntax

Collectors.groupingBy(Function classifier)

Example

```
import java.util.*;
import java.util.stream.*;

class Employee {
   String name;
   String dept;
   double salary;

   Employee(String name, String dept, double salary) {
      this.name = name;
      this.dept = dept;
      this.salary = salary;
   }

   public String getDept() { return dept; }
}

public class GroupingExample {
   public static void main(String[] args) {
      List<Employee> employees = Arrays.asList(
         new Employee("A", "IT", 50000),
         new Employee("B", "HR", 40000),
         new Employee("C", "IT", 60000)
      );

      Map<String, List<Employee>> result =
         employees.stream()
               .collect(Collectors.groupingBy(Employee::getDept));

      System.out.println(result);
   }
}
```

✔ Key = Department

✔ Value = List of Employees

✅ 2. Multi-level grouping example

Group by department → then by salary range.

```
Map<String, Map<String, List<Employee>>> result =
```

```
employees.stream()
        .collect(Collectors.groupingBy(
            Employee::getDept,
            Collectors.groupingBy(e ->
                e.salary > 50000 ? "HIGH" : "LOW"
            )
        ));

System.out.println(result);
```

✔ Nested Map structure

✔ Very common interview question

✅ 3. Difference between groupingBy() and partitioningBy()

groupingBy	partitioningBy
Any key type	Boolean key
Multiple groups	Only 2 groups
More flexible	Simpler

Example:

```
Collectors.partitioningBy(e -> e.salary > 50000);
```

Returns:

```
Map<Boolean, List<Employee>>
```

✅ 4. How to count elements in each group?

```
Map<String, Long> count =
    employees.stream()
        .collect(Collectors.groupingBy(
            Employee::getDept,
            Collectors.counting()
        ));

System.out.println(count);
```

✔ Downstream collector = counting()

✅ 5. How to sum values group-wise?

```
Map<String, Double> sum =
   employees.stream()
         .collect(Collectors.groupingBy(
            Employee::getDept,
            Collectors.summingDouble(e -> e.salary)
         ));

System.out.println(sum);
```

✅ 6. How to find max salary employee per department?

```
Map<String, Optional<Employee>> maxSalary =
   employees.stream()
         .collect(Collectors.groupingBy(
            Employee::getDept,
            Collectors.maxBy(
               Comparator.comparing(e -> e.salary)
            )
         ));

System.out.println(maxSalary);
```

✔ Returns Optional

✔ Common senior-level question

✅ 7. How to flatten nested collections?

```
List<List<Integer>> nested = Arrays.asList(
   Arrays.asList(1,2),
   Arrays.asList(3,4)
);

List<Integer> flat =
   nested.stream()
       .flatMap(List::stream)
       .collect(Collectors.toList());

System.out.println(flat);
```

✅ 8. How to collect into Set?

```
Set<String> set =
```

```
   employees.stream()
         .map(e -> e.name)
         .collect(Collectors.toSet());
```

✅ 9. How to collect into LinkedList?

```
LinkedList<String> list =
   employees.stream()
         .map(e -> e.name)
         .collect(Collectors.toCollection(LinkedList::new));
```

✔ toCollection() allows custom collection type

✅ 10. How to create custom Collector?

Example: Collect into StringBuilder

```
Collector<String, StringBuilder, String> customCollector =
   Collector.of(
      StringBuilder::new,         // supplier
      StringBuilder::append,      // accumulator
      StringBuilder::append,      // combiner
      StringBuilder::toString     // finisher
   );

String result = Stream.of("A","B","C")
                  .collect(customCollector);

System.out.println(result);
```

✔ Important for senior interviews

✅ 11. What is downstream collector?

Collector applied after grouping.

Example:

```
Collectors.groupingBy(
   Employee::getDept,
   Collectors.counting()
)
```

Here counting() is downstream collector.

✅ 12. How does mapping() work?

Used to transform values inside grouping.

```
Map<String, List<String>> namesByDept =
   employees.stream()
         .collect(Collectors.groupingBy(
            Employee::getDept,
            Collectors.mapping(
               e -> e.name,
               Collectors.toList()
            )
         ));
```

✔ mapping() transforms grouped values

✅ 13. What is reducing() collector?

Performs reduction inside grouping.

```
Map<String, Double> totalSalary =
   employees.stream()
         .collect(Collectors.groupingBy(
            Employee::getDept,
            Collectors.reducing(
               0.0,
               e -> e.salary,
               Double::sum
            )
         ));
```

✅ 14. What is teeing() collector (Java 12)?

Combines two collectors.

Example: Get average and count together.

```
var result =
   employees.stream()
         .collect(Collectors.teeing(
            Collectors.counting(),
```

```
            Collectors.averagingDouble(e -> e.salary),
            (count, avg) -> "Count: " + count + ", Avg: " + avg
        ));

System.out.println(result);
```

✔ Powerful advanced feature

✅ 15. Immutable collectors in Java 10+

```
List<String> immutable =
   employees.stream()
         .map(e -> e.name)
         .collect(Collectors.toUnmodifiableList());
```

Also:

- toUnmodifiableSet()
- toUnmodifiableMap()

✅ 16. How to sort Map using stream?

```
Map<String, Double> sorted =
   sum.entrySet()
     .stream()
     .sorted(Map.Entry.comparingByValue())
     .collect(Collectors.toMap(
        Map.Entry::getKey,
        Map.Entry::getValue,
        (a,b) -> a,
        LinkedHashMap::new
     ));
```

✔ LinkedHashMap preserves order

✅ 17. How to group by multiple fields?

Example: Group by dept and salary

```
Map<String, Map<Double, List<Employee>>> result =
   employees.stream()
```

```
        .collect(Collectors.groupingBy(
            Employee::getDept,
            Collectors.groupingBy(e -> e.salary)
        ));
```

Or create composite key class.

✅ 18. Convert List to Map<Key,List>

```
Map<String, List<Employee>> map =
    employees.stream()
        .collect(Collectors.groupingBy(
            e -> e.dept
        ));
```

General pattern:

```
Collectors.groupingBy(keyExtractor)
```

✅ 19. What is concurrent collector?

Collector that supports parallel execution.

Example:

```
ConcurrentMap<String, List<Employee>> map =
    employees.parallelStream()
        .collect(Collectors.groupingByConcurrent(
            Employee::getDept
        ));
```

✔ Improves parallel performance

✔ No ordering guarantee

✅ 20. How to collect in parallel safely?

Rules:

- Use stateless lambdas
- Use thread-safe collectors
- Avoid shared mutable state

Example:

```
ConcurrentMap<String, Long> result =
   employees.parallelStream()
         .collect(Collectors.groupingByConcurrent(
            Employee::getDept,
            Collectors.counting()
         ));
```

Final Interview Insights

✔ groupingBy is most asked

✔ Multi-level grouping is senior-level

✔ Downstream collectors are critical

✔ reducing vs summing difference

✔ teeing is advanced

✔ groupingByConcurrent for parallel

✔ Always handle Optional

If you want next:

✅ 25 Advanced real-time Stream coding problems
✅ Custom Collector deep internal explanation
✅ Parallel Stream internal working (ForkJoinPool diagram explanation)
✅ 50 FAANG-level tricky Stream questions

✅ 5. Parallel Streams (15 Questions)

1. What is parallel stream?
2. How parallel stream works internally?
3. What is ForkJoinPool?
4. When should we use parallel streams?
5. When should we NOT use parallel streams?
6. Performance comparison: loop vs stream vs parallel stream
7. Thread safety in parallel streams
8. What is Spliterator?
9. What is characteristics of Spliterator?
10. How ordering affects performance?
11. How to control parallelism?
12. What is combiner in parallel stream?

13. Why reduce needs associative function?
14. What is non-interference?
15. Stateful lambda problem in parallel stream?

Now we are entering **Advanced Level – Parallel Streams (Very Important for Senior Interviews)**.

I will explain each concept with:

- ✔ Clear explanation
- ✔ Internal working
- ✔ Java code
- ✔ Interview insight

✅ 1. What is Parallel Stream?

🔹 Definition

A **parallel stream** divides data into multiple chunks and processes them **simultaneously using multiple threads**.

Created using:

parallelStream()

or

stream().parallel()

Example

```
import java.util.*;
import java.util.stream.*;

public class ParallelExample {
   public static void main(String[] args) {
      List<Integer> list = Arrays.asList(1,2,3,4,5,6,7,8);

      list.parallelStream()
         .forEach(n ->
            System.out.println(Thread.currentThread().getName() + " -> " + n)
         );
   }
}
```

✔ Output order not guaranteed

✔ Uses multiple threads

✅ 2. How Parallel Stream Works Internally?

Internally it uses:

1. **Spliterator** → Splits data into chunks
2. **ForkJoinPool (Common Pool)** → Executes tasks
3. **Combiner** → Merges partial results

Flow:

Source → Splitting → Parallel Execution → Combining → Result

Example reduction:

```
int sum = IntStream.range(1, 10)
            .parallel()
            .sum();
```

Internally:

- Range split into chunks
- Each chunk summed independently
- Partial sums combined

✅ 3. What is ForkJoinPool?

Definition

A special thread pool introduced in Java 7 for **divide and conquer parallelism**.

Used automatically by parallel streams.

Example – Custom ForkJoinPool

```
import java.util.concurrent.*;

public class CustomPoolExample {
   public static void main(String[] args) throws Exception {
```

```
    ForkJoinPool pool = new ForkJoinPool(4);

    pool.submit(() ->
      IntStream.range(1,10)
            .parallel()
            .forEach(System.out::println)
    ).get();

    pool.shutdown();
  }
}
```

✔ Uses work-stealing algorithm

✔ Default pool size = CPU cores

✅ 4. When Should We Use Parallel Streams?

✔ Large data sets

✔ CPU-intensive operations

✔ Stateless operations

✔ Independent tasks

Example:

```
List<Integer> list = IntStream.range(1, 1_000_000)
                    .boxed()
                    .collect(Collectors.toList());

long count = list.parallelStream()
          .filter(n -> n % 2 == 0)
          .count();
```

✅ 5. When Should We NOT Use Parallel Streams?

❌ Small data
❌ I/O operations
❌ Shared mutable state
❌ Order-sensitive tasks

Bad Example:

```
list.parallelStream()
  .forEach(n -> System.out.println(n)); // order not guaranteed
```

✅ 6. Performance Comparison: Loop vs Stream vs Parallel Stream

Sequential Loop

```
long start = System.currentTimeMillis();

long sum = 0;
for (int i = 1; i <= 1_000_000; i++) {
   sum += i;
}

long end = System.currentTimeMillis();
System.out.println("Loop Time: " + (end-start));
```

Stream

```
long start = System.currentTimeMillis();

long sum = IntStream.rangeClosed(1,1_000_000).sum();

long end = System.currentTimeMillis();
System.out.println("Stream Time: " + (end-start));
```

Parallel Stream

```
long start = System.currentTimeMillis();

long sum = IntStream.rangeClosed(1,1_000_000)
                .parallel()
                .sum();

long end = System.currentTimeMillis();
System.out.println("Parallel Time: " + (end-start));
```

✔ Parallel may be faster for large CPU tasks

✔ Overhead exists

✅ 7. Thread Safety in Parallel Streams

Streams are NOT inherently thread-safe.

Correct:

```
List<Integer> result =
   list.parallelStream()
```

```
    .map(n -> n * 2)
    .collect(Collectors.toList());
```

Wrong:

```
List<Integer> result = new ArrayList<>();

list.parallelStream()
  .forEach(result::add); // NOT thread-safe
```

Fix:

```
List<Integer> result =
  list.parallelStream()
    .collect(Collectors.toList());
```

✅ 8. What is Spliterator?

□ Definition

Special iterator that supports parallel traversal.

Responsibilities:

- Splitting
- Traversing
- Reporting characteristics

Example:

```
Spliterator<Integer> spliterator = list.spliterator();
System.out.println(spliterator.estimateSize());
```

✅ 9. Characteristics of Spliterator

Common characteristics:

- ORDERED
- DISTINCT
- SORTED
- SIZED
- NONNULL
- IMMUTABLE
- CONCURRENT

Example:

```
int characteristics = spliterator.characteristics();
System.out.println(characteristics);
```

✅ 10. How Ordering Affects Performance?

Ordered streams:

```
list.parallelStream()
   .forEachOrdered(System.out::println);
```

Maintaining order:

- Slower
- Less parallel optimization

Unordered:

```
list.parallelStream()
   .unordered()
   .limit(5)
   .forEach(System.out::println);
```

✔ Faster in parallel

✅ 11. How to Control Parallelism?

Change common pool size

```
System.setProperty(
   "java.util.concurrent.ForkJoinPool.common.parallelism",
   "4"
);
```

Or use custom ForkJoinPool (shown earlier).

✅ 12. What is Combiner in Parallel Stream?

Combiner merges partial results.

Example using reduce:

```
int sum = list.parallelStream()
        .reduce(0,
            (a,b) -> a + b,   // accumulator
            (a,b) -> a + b);  // combiner
```

Accumulator = combine element
Combiner = merge partial results

✅ 13. Why Reduce Needs Associative Function?

Because:

Parallel execution:

(a + b) + c = a + (b + c)

Associativity required for correctness.

Correct:

(a,b) -> a + b

Wrong:

(a,b) -> a - b   // NOT associative

✅ 14. What is Non-Interference?

Stream operations must NOT modify source.

Wrong:

```
list.parallelStream()
   .forEach(n -> list.add(n));  // ConcurrentModificationException
```

Correct:

- Treat source as immutable

✅ 16. Stateful Lambda Problem in Parallel Stream

Stateful lambda depends on shared mutable state.

Wrong Example:

```
List<Integer> result = new ArrayList<>();

list.parallelStream()
   .map(n -> {
      result.add(n);  // unsafe
      return n;
   })
   .count();
```

Problem:

- Race conditions
- Inconsistent results

Correct approach:

```
List<Integer> result =
   list.parallelStream()
      .collect(Collectors.toList());
```

□ Senior-Level Interview Summary

✔ Parallel streams use ForkJoinPool

✔ Spliterator splits data

✔ Combiner merges results

✔ Reduce requires associative function

✔ Avoid stateful lambdas

✔ Ordering reduces performance

✔ Use groupingByConcurrent for parallel

If you want next:

✅ Full internal architecture diagram explanation
✅ Custom Spliterator implementation
✅ 30 Advanced parallel stream coding problems

✅ Streams vs CompletableFuture comparison
✅ FAANG tricky parallel stream questions

✅ 6. Functional Interface & Lambda (15 Questions)

1. What functional interfaces are used in Streams?
2. What is Predicate?
3. What is Function?
4. What is Consumer?
5. What is Supplier?
6. What is BinaryOperator?
7. What is UnaryOperator?
8. What is method reference?
9. What is target typing?
10. What is effectively final variable?
11. Lambda vs anonymous class?
12. How streams use functional interfaces internally?
13. What is composition in Function?
14. What is chaining in Predicate?
15. What happens if lambda throws exception?

Now we are covering **Functional Interfaces & Lambda in Streams (Very Important for Interviews)**.

I'll explain each with:

- ✔ Concept
- ✔ Java syntax
- ✔ Stream usage
- ✔ Interview insight

✅ 1. What functional interfaces are used in Streams?

Streams heavily use **java.util.function** package.

Main functional interfaces:

Interface	Used In
Predicate	filter(), allMatch()
Function	map()

Consumer	forEach()
Supplier	Stream.generate()
BinaryOperator	reduce()
UnaryOperator	map()
Comparator	sorted()

Example usage in Stream:

```
list.stream()
   .filter(n -> n > 2)   // Predicate
   .map(n -> n * 2)      // Function
   .forEach(System.out::println); // Consumer
```

✅ 2. What is Predicate?

□ Definition

Represents a boolean condition.

```
@FunctionalInterface
public interface Predicate<T> {
   boolean test(T t);
}
```

Example

```
Predicate<Integer> isEven = n -> n % 2 == 0;

System.out.println(isEven.test(4)); // true
```

Used in Stream

```
list.stream()
   .filter(n -> n % 2 == 0)
   .forEach(System.out::println);
```

✅ 3. What is Function?

□ Definition

Takes input and returns output.

```
@FunctionalInterface
public interface Function<T,R> {
   R apply(T t);
```

```
}
```

Example

```
Function<String, Integer> length = s -> s.length();

System.out.println(length.apply("Java")); // 4
```

Used in Stream

```
list.stream()
   .map(n -> n * 2)
   .forEach(System.out::println);
```

✅ 4. What is Consumer?

□ Definition

Consumes input, returns nothing.

```
@FunctionalInterface
public interface Consumer<T> {
   void accept(T t);
}
```

Example

```
Consumer<String> print = s -> System.out.println(s);
print.accept("Hello");
```

Used in Stream

```
list.stream()
   .forEach(System.out::println);
```

✅ 5. What is Supplier?

□ Definition

Supplies value, no input.

```
@FunctionalInterface
public interface Supplier<T> {
   T get();
```

```
}
```

Example

```
Supplier<Double> random = () -> Math.random();
System.out.println(random.get());
```

Used in Stream

```
Stream.generate(() -> "Hello")
    .limit(3)
    .forEach(System.out::println);
```

✅ 6. What is BinaryOperator?

▯ Definition

Takes two same-type inputs and returns same type.

```
BinaryOperator<Integer> sum = (a, b) -> a + b;
```

Used in reduce()

```
int result = list.stream()
            .reduce(0, (a,b) -> a + b);
```

✅ 7. What is UnaryOperator?

▯ Definition

Takes one input and returns same type.

```
UnaryOperator<Integer> square = n -> n * n;
```

Used in map()

```
list.stream()
   .map(n -> n * 2)
   .forEach(System.out::println);
```

✅ 8. What is Method Reference?

Shortcut of lambda when method already exists.

Types:

1. Static method reference
2. Instance method reference
3. Constructor reference

Example

Lambda:

```
list.stream().forEach(n -> System.out.println(n));
```

Method reference:

```
list.stream().forEach(System.out::println);
```

Constructor reference:

```
Supplier<List<String>> supplier = ArrayList::new;
```

✅ 9. What is Target Typing?

Compiler determines lambda type based on context.

Example:

```
Predicate<Integer> p = n -> n > 5;
```

Here:

- Compiler knows n is Integer
- Because target type = Predicate

✅ 10. What is Effectively Final Variable?

Local variable used inside lambda must not change.

Correct:

```
int x = 10;
list.stream().forEach(n -> System.out.println(n + x));
```

Wrong:

```
int x = 10;
x++; // Not allowed if used in lambda
```

✅ 11. Lambda vs Anonymous Class

Lambda	Anonymous Class
Less boilerplate	Verbose
No new scope for this	Has its own this
Functional interface only	Any interface

Anonymous Class:

```
Runnable r = new Runnable() {
   public void run() {
      System.out.println("Hello");
   }
};
```

Lambda:

```
Runnable r = () -> System.out.println("Hello");
```

✅ 12. How Streams Use Functional Interfaces Internally?

Examples:

Stream Method	Functional Interface
filter()	Predicate
map()	Function
forEach()	Consumer
reduce()	BinaryOperator
generate()	Supplier

Example:

```
list.stream()
   .filter(n -> n > 2)   // Predicate
   .map(n -> n * 2)      // Function
   .forEach(System.out::println); // Consumer
```

✅ 13. What is Composition in Function?

Combine functions.

```
Function<Integer, Integer> multiply = n -> n * 2;
Function<Integer, Integer> add = n -> n + 3;

Function<Integer, Integer> combined =
   multiply.andThen(add);

System.out.println(combined.apply(5)); // (5*2)+3 = 13
```

Also:

```
multiply.compose(add);
```

✅ 14. What is Chaining in Predicate?

Combine multiple conditions.

```
Predicate<Integer> greaterThan2 = n -> n > 2;
Predicate<Integer> even = n -> n % 2 == 0;

Predicate<Integer> combined =
   greaterThan2.and(even);

list.stream()
   .filter(combined)
   .forEach(System.out::println);
```

Also:

- or()
- negate()

✅ 15. What Happens If Lambda Throws Exception?

Checked exceptions not allowed directly.

Wrong:

```
list.stream()
   .forEach(n -> {
      throw new Exception(); // compile error
   });
```

Correct (wrap in RuntimeException):

```
list.stream()
```

```
.forEach(n -> {
    try {
        if(n == 2)
            throw new Exception();
    } catch(Exception e) {
        throw new RuntimeException(e);
    }
});
```

Or create custom wrapper method.

□ Senior-Level Interview Summary

✔ Streams built entirely on functional interfaces

✔ Predicate → filtering

✔ Function → transformation

✔ Consumer → side effect

✔ Supplier → generation

✔ BinaryOperator → reduction

✔ Associativity required for reduce

✔ Lambdas must use effectively final variables

✔ Avoid throwing checked exceptions directly

If you want next:

✅ 30 Real Interview Coding Questions using Functional Interfaces
✅ Deep Internal Working of Stream Pipeline
✅ Custom Functional Interface Implementation
✅ Streams + Optional Advanced Guide
✅ FAANG-Level Tricky Lambda Questions

✅ 7. Real-Time Scenario Based (20 Questions)

1. Find duplicate elements in list
2. Find first non-repeated character
3. Find frequency of each character

4. Sort employees by salary
5. Find second highest salary
6. Find highest paid employee in each department
7. Count male/female employees
8. Find average salary by department
9. Merge two lists and remove duplicates
10. Convert List to Map<String,Integer>
11. Reverse each string in list
12. Flatten List<List>
13. Partition numbers even/odd
14. Check if list contains duplicates
15. Find longest string
16. Convert list to comma separated string
17. Find missing number in array
18. Find top 3 highest numbers
19. Group anagrams together
20. Sort map by values

Now we are solving **20 Real Interview Coding Problems using Java Streams (Detailed + Clean Code + Interview-Level Solutions)**.

I'll include:

- ✔ Problem
- ✔ Java 8 Stream solution
- ✔ Explanation
- ✔ Interview tip

✅ 1. Find Duplicate Elements in List

```
import java.util.*;
import java.util.stream.*;

public class DuplicateExample {
   public static void main(String[] args) {
      List<Integer> list = Arrays.asList(1,2,3,2,4,1,5);

      Set<Integer> duplicates = list.stream()
            .filter(n -> Collections.frequency(list, n) > 1)
            .collect(Collectors.toSet());

      System.out.println(duplicates); // [1,2]
   }
}
```

✔ Uses frequency()

✔ Convert to Set to avoid repeating duplicates

✅ 2. Find First Non-Repeated Character

```
String str = "swiss";

Character result = str.chars()
      .mapToObj(c -> (char) c)
      .filter(c -> str.indexOf(c) == str.lastIndexOf(c))
      .findFirst()
      .orElse(null);

System.out.println(result); // w
```

✔ Maintains order

✔ Uses Optional

✅ 3. Find Frequency of Each Character

```
String str = "banana";

Map<Character, Long> frequency =
      str.chars()
        .mapToObj(c -> (char) c)
        .collect(Collectors.groupingBy(
            c -> c,
            Collectors.counting()
        ));

System.out.println(frequency);
```

✔ Most asked interview question

✅ 4. Sort Employees by Salary

```
employees.stream()
      .sorted(Comparator.comparing(Employee::getSalary))
```

```
    .forEach(e -> System.out.println(e.getSalary()));
```

Descending:

```
.sorted(Comparator.comparing(Employee::getSalary).reversed())
```

✅ 5. Find Second Highest Salary

```
Double secondHighest =
    employees.stream()
        .map(Employee::getSalary)
        .sorted(Comparator.reverseOrder())
        .skip(1)
        .findFirst()
        .orElse(null);

System.out.println(secondHighest);
```

✔ Uses skip(1)

✅ 6. Highest Paid Employee in Each Department

```
Map<String, Optional<Employee>> result =
    employees.stream()
        .collect(Collectors.groupingBy(
            Employee::getDepartment,
            Collectors.maxBy(
                Comparator.comparing(Employee::getSalary)
            )
        ));

System.out.println(result);
```

✔ Very important grouping question

✅ 7. Count Male/Female Employees

```
Map<String, Long> genderCount =
    employees.stream()
        .collect(Collectors.groupingBy(
            Employee::getGender,
            Collectors.counting()
        ));
```

```
System.out.println(genderCount);
```

✅ 8. Average Salary by Department

```
Map<String, Double> avgSalary =
    employees.stream()
        .collect(Collectors.groupingBy(
            Employee::getDepartment,
            Collectors.averagingDouble(Employee::getSalary)
        ));

System.out.println(avgSalary);
```

✅ 9. Merge Two Lists and Remove Duplicates

```
List<Integer> list1 = Arrays.asList(1,2,3);
List<Integer> list2 = Arrays.asList(3,4,5);

List<Integer> merged =
    Stream.concat(list1.stream(), list2.stream())
        .distinct()
        .collect(Collectors.toList());

System.out.println(merged);
```

✅ 10. Convert List to Map<String,Integer>

```
List<String> names = Arrays.asList("A","BB","CCC");

Map<String, Integer> map =
    names.stream()
        .collect(Collectors.toMap(
            s -> s,
            s -> s.length()
        ));

System.out.println(map);
```

✅ 11. Reverse Each String in List

```
List<String> reversed =
    names.stream()
```

```
        .map(s -> new StringBuilder(s).reverse().toString())
        .collect(Collectors.toList());

System.out.println(reversed);
```

✅ 12. Flatten List<List>

```
List<List<Integer>> nested = Arrays.asList(
    Arrays.asList(1,2),
    Arrays.asList(3,4)
);

List<Integer> flat =
    nested.stream()
        .flatMap(List::stream)
        .collect(Collectors.toList());

System.out.println(flat);
```

✅ 13. Partition Numbers Even/Odd

```
Map<Boolean, List<Integer>> partition =
    list.stream()
      .collect(Collectors.partitioningBy(n -> n % 2 == 0));

System.out.println(partition);
```

✔ True → Even

✔ False → Odd

✅ 14. Check If List Contains Duplicates

```
boolean hasDuplicates =
    list.stream()
      .distinct()
      .count() != list.size();

System.out.println(hasDuplicates);
```

✅ 15. Find Longest String

```
String longest =
    names.stream()
        .max(Comparator.comparing(String::length))
        .orElse(null);

System.out.println(longest);
```

✅ 16. Convert List to Comma Separated String

```
String result =
    names.stream()
        .collect(Collectors.joining(", "));

System.out.println(result);
```

✅ 17. Find Missing Number in Array (1 to N)

```
List<Integer> numbers = Arrays.asList(1,2,4,5);

int n = 5;

int expectedSum = IntStream.rangeClosed(1, n).sum();
int actualSum = numbers.stream().mapToInt(Integer::intValue).sum();

System.out.println(expectedSum - actualSum); // 3
```

✅ 18. Find Top 3 Highest Numbers

```
list.stream()
  .sorted(Comparator.reverseOrder())
  .limit(3)
  .forEach(System.out::println);
```

✅ 19. Group Anagrams Together

```
List<String> words = Arrays.asList("eat","tea","tan","ate","nat","bat");

Map<String, List<String>> grouped =
    words.stream()
        .collect(Collectors.groupingBy(word -> {
            char[] chars = word.toCharArray();
            Arrays.sort(chars);
            return new String(chars);
        }));
```

```
System.out.println(grouped);
```

✔ Very popular interview problem

✅ 20. Sort Map by Values

```
Map<String, Integer> map = Map.of("A",3,"B",1,"C",2);

LinkedHashMap<String, Integer> sorted =
    map.entrySet()
      .stream()
      .sorted(Map.Entry.comparingByValue())
      .collect(Collectors.toMap(
         Map.Entry::getKey,
         Map.Entry::getValue,
         (a,b) -> a,
         LinkedHashMap::new
      ));

System.out.println(sorted);
```

✔ Use LinkedHashMap to preserve order

□ Interview Master Tips

✔ Always use Optional safely

✔ Prefer groupingBy + downstream collectors

✔ Avoid stateful lambda

✔ Use mapToInt for performance

✔ Use distinct() for duplicate removal

✔ For top N → sorted().limit()

✔ For merging lists → Stream.concat()

If you want next:

✅ 30 Advanced FAANG-Level Stream Coding Problems
✅ Tricky Output-Based Stream Questions
✅ Stream Performance Benchmark Guide
✅ Custom Collector Advanced Implementation

✅ 8. Advanced & Internal Working (20 Questions)

1. How stream pipeline is executed internally?
2. What is stream source?
3. What is stream sink?
4. How lazy evaluation improves performance?
5. What is Spliterator splitting mechanism?
6. How short-circuiting works internally?
7. How reduce works internally?
8. What is difference between stream and reactive streams?
9. Can streams handle infinite data?
10. What is infinite stream example?
11. What is Stream.iterate()?
12. What is Stream.generate()?
13. How memory is managed in streams?
14. Are streams thread-safe?
15. What is stateless lambda?
16. Why streams are not data structure?
17. How to debug stream?
18. What is Stream.Builder?
19. How to create custom stream?
20. What are limitations of Streams API?

Now we're going into **Deep Internal Architecture of Java Streams (Senior / Architect Level Understanding)**.

This section explains **how Streams work internally**, not just usage.

✅ 1. How Stream Pipeline is Executed Internally?

🔹 Pipeline Structure

A stream pipeline has:

Source → Intermediate Operations → Terminal Operation

Example:

```
List<Integer> result = list.stream()
    .filter(n -> n > 2)
    .map(n -> n * 2)
    .collect(Collectors.toList());
```

⚙️ Internal Execution Steps

1. stream() creates a pipeline object.
2. filter() and map() create intermediate pipeline stages.
3. No data processed yet (lazy).
4. collect() triggers execution.
5. Elements flow one-by-one through the pipeline.

🧠 Important Concept

Streams use **pull model**:

- Terminal operation pulls elements from source.
- Each element passes through full pipeline before next element.

Equivalent internal idea:

```
for(Integer n : list) {
   if(n > 2) {
      int mapped = n * 2;
      result.add(mapped);
   }
}
```

But optimized internally.

✅ 2. What is Stream Source?

📘 Definition

The origin of stream data.

Common sources:

- Collection
- Array
- File
- Generator
- Infinite stream

Example:

```
Stream<Integer> s1 = list.stream();
Stream<String> s2 = Stream.of("A","B");
```

```
IntStream s3 = IntStream.range(1,5);
```

✅ 3. What is Stream Sink?

📘 Definition

The final consumer of elements in pipeline.

Sink = Terminal operation.

Example:

```
list.stream()
   .filter(n -> n > 2)
   .forEach(System.out::println);  // Sink
```

Internally:

- Each stage wraps next stage
- Terminal operation consumes elements

✅ 4. How Lazy Evaluation Improves Performance?

Intermediate operations don't run until terminal operation.

Example:

```
list.stream()
   .filter(n -> {
      System.out.println("Filtering " + n);
      return n > 2;
   })
   .map(n -> {
      System.out.println("Mapping " + n);
      return n * 2;
   })
   .findFirst();
```

Execution stops once first match found.

✔ No unnecessary processing

✔ Works element-by-element

✔ Reduces memory overhead

✅ 5. What is Spliterator Splitting Mechanism?

▯ Definition

Spliterator splits data into chunks for parallel processing.

Internal steps:

trySplit() → divide data
process in parallel
combine results

Example:

```
Spliterator<Integer> spliterator = list.spliterator();
Spliterator<Integer> split = spliterator.trySplit();
```

✔ Used only in parallel streams

✔ Enables divide-and-conquer

✅ 6. How Short-Circuiting Works Internally?

Short-circuit terminal operations:

- findFirst()
- findAny()
- anyMatch()
- allMatch()
- noneMatch()
- limit()

Example:

```
list.stream()
   .filter(n -> n > 2)
   .findFirst();
```

Internally:

- Stops once condition satisfied
- No further elements processed

Equivalent idea:

```
for(Integer n : list) {
   if(n > 2) {
      return n;
   }
}
```

✅ 7. How reduce() Works Internally?

Example:

```
int sum = list.stream()
           .reduce(0, (a,b) -> a + b);
```

Internal logic:

```
int result = 0;
for(Integer n : list) {
   result = result + n;
}
```

Parallel version:

1. Split list
2. Compute partial sums
3. Combine using combiner

Important:

✔ Must use associative function

✅ 8. Difference Between Stream and Reactive Streams

Stream	Reactive Stream
Pull model	Push model
Synchronous	Asynchronous
No backpressure	Supports backpressure
Java 8 built-in	Project Reactor, RxJava

Stream example:

```
list.stream().map(n -> n * 2);
```

Reactive example (conceptual):

```
Flux.fromIterable(list)
```

```
    .map(n -> n * 2)
    .subscribe(System.out::println);
```

Reactive is non-blocking.

✅ 9. Can Streams Handle Infinite Data?

Yes, but must use short-circuit operation.

Example:

```
Stream.iterate(1, n -> n + 1)
    .limit(5)
    .forEach(System.out::println);
```

Without limit → infinite loop.

✅ 10. Infinite Stream Example

```
Stream.generate(Math::random)
    .limit(3)
    .forEach(System.out::println);
```

Or:

```
IntStream.iterate(1, n -> n + 1)
      .limit(10)
      .forEach(System.out::println);
```

✅ 11. What is Stream.iterate()?

Creates infinite sequential stream.

```
Stream.iterate(1, n -> n + 2)
    .limit(5)
    .forEach(System.out::println);
```

Java 9 version:

```
Stream.iterate(1, n -> n < 10, n -> n + 1);
```

✅ 12. What is Stream.generate()?

Creates infinite stream using Supplier.

```
Stream.generate(() -> "Hello")
    .limit(3)
    .forEach(System.out::println);
```

✅ 13. How Memory is Managed in Streams?

✔ Streams do NOT store elements

✔ They process element-by-element

✔ Stateless operations use minimal memory

✔ Stateful operations (sorted, distinct) buffer elements

Example:

```
list.stream().sorted(); // needs full buffering
```

✅ 14. Are Streams Thread-Safe?

Sequential stream:

- Safe

Parallel stream:

- Safe internally
- NOT safe if using shared mutable state

Wrong:

```
List<Integer> result = new ArrayList<>();

list.parallelStream()
   .forEach(result::add);  // unsafe
```

Correct:

```
list.parallelStream()
   .collect(Collectors.toList());
```

✅ 15. What is Stateless Lambda?

Lambda that does NOT depend on external mutable state.

Good:

```
list.stream().map(n -> n * 2);
```

Bad:

```
int sum = 0;
list.stream().forEach(n -> sum += n); // stateful
```

✅ 16. Why Streams Are Not Data Structure?

Because:

- They do not store elements
- They don't support random access
- They cannot be reused

Stream = View of data

✅ 17. How to Debug Stream?

Using peek():

```
list.stream()
   .filter(n -> n > 2)
   .peek(n -> System.out.println("Filtered: " + n))
   .map(n -> n * 2)
   .collect(Collectors.toList());
```

Or split into steps.

✅ 18. What is Stream.Builder?

Used to build stream manually.

```
Stream.Builder<String> builder = Stream.builder();

builder.add("A");
builder.add("B");

Stream<String> stream = builder.build();

stream.forEach(System.out::println);
```

✅ 19. How to Create Custom Stream?

Using StreamSupport:

```
Stream<Integer> customStream =
   StreamSupport.stream(list.spliterator(), false);

customStream.forEach(System.out::println);
```

Or create custom Spliterator.

✅ 20. Limitations of Streams API

✔ Not good for complex stateful logic

✔ Hard to debug

✔ Not ideal for heavy I/O

✔ Parallel streams share common pool

✔ No checked exception handling

✔ Cannot reuse stream

□ Architect-Level Summary

✔ Streams use pipeline model

✔ Spliterator handles splitting

✔ Terminal operation triggers execution

✔ Lazy evaluation improves performance

✔ Parallel streams use ForkJoinPool

✔ Must use stateless lambdas

✔ Streams are pull-based, reactive is push-based

If you want next:

✅ Full Internal Diagram Explanation (Pipeline + Spliterator)
✅ Custom Spliterator Implementation

✅ 40 FAANG-Level Advanced Questions
✅ Stream vs CompletableFuture Deep Comparison

✅ 9. Coding Output-Based Questions (Bonus 15)

1. What is output of chained map().filter().count()?
2. What happens if map() returns null?
3. What if reduce() identity is wrong?
4. What if parallelStream() modifies shared variable?
5. What if no terminal operation is used?
6. What if findFirst() used in parallel?
7. What if collector is not thread safe?
8. What happens with duplicate key in toMap()?
9. What if sorted comparator inconsistent?
10. What if stream source is modified?
11. What if stream is consumed twice?
12. What if flatMap returns null?
13. What if infinite stream without limit?
14. What if order is removed?
15. What if combiner is incorrect?

Excellent
Now we are entering **Tricky / Output-Based / Edge Case Stream Questions** — exactly what interviewers love.

I'll explain:

- ✔ Code
- ✔ Output / Exception
- ✔ Why it happens internally
- ✔ Interview insight

✅ 1. What is output of chained map().filter().count()?

```
import java.util.*;
import java.util.stream.*;

public class Test1 {
   public static void main(String[] args) {
```

```
    List<Integer> list = Arrays.asList(1,2,3,4,5);

    long result = list.stream()
        .map(n -> {
            System.out.println("Mapping: " + n);
            return n * 2;
        })
        .filter(n -> {
            System.out.println("Filtering: " + n);
            return n > 5;
        })
        .count();

    System.out.println("Count: " + result);
  }
}
```

Output Pattern (Order important)

```
Mapping: 1
Filtering: 2
Mapping: 2
Filtering: 4
Mapping: 3
Filtering: 6
Mapping: 4
Filtering: 8
Mapping: 5
Filtering: 10
Count: 3
```

Why?

Streams process **element-by-element**, not stage-by-stage.

Equivalent logic:

```
for(Integer n : list){
  int mapped = n * 2;
  if(mapped > 5){
    count++;
  }
}
```

✔ Lazy evaluation

✔ Pipeline execution

✅ 2. What happens if map() returns null?

```
List<String> list = Arrays.asList("A","B","C");

list.stream()
   .map(s -> null)
   .forEach(System.out::println);
```

Output:
null
null
null

✔ No exception here.

BUT:

```
list.stream()
   .map(s -> null)
   .sorted()
   .forEach(System.out::println);
```

❌ **Throws:**
NullPointerException

Because sorted() cannot compare null.

✅ 3. What if reduce() identity is wrong?

Correct:

```
int sum = list.stream().reduce(0, Integer::sum);
```

Wrong identity:

```
int sum = list.stream().reduce(10, Integer::sum);
System.out.println(sum);
```

If list = [1,2,3]

Output:

16

Why?

10 + 1 + 2 + 3

✔ Identity participates in computation

✔ Must be neutral element

For sum → identity = 0
For multiplication → identity = 1

✅ 4. What if parallelStream() modifies shared variable?

Wrong:

```
List<Integer> list = Arrays.asList(1,2,3,4,5);
int[] sum = {0};

list.parallelStream()
   .forEach(n -> sum[0] += n);

System.out.println(sum[0]);
```

Possible output:

Unpredictable (race condition)

Why?

- Multiple threads modify same memory

Correct:

```
int sum = list.parallelStream()
          .reduce(0, Integer::sum);
```

✔ Avoid shared mutable state

✅ 5. What if no terminal operation is used?

```
list.stream()
   .filter(n -> {
      System.out.println(n);
      return n > 2;
   });
```

Output:

Nothing.

✔ Lazy execution

✔ No terminal → No execution

✅ 6. What if findFirst() used in parallel?

```
List<Integer> list = Arrays.asList(1,2,3,4,5);

list.parallelStream()
   .findFirst()
   .ifPresent(System.out::println);
```

Output:

1

✔ Maintains encounter order

✔ Slower in parallel

If use:

```
list.parallelStream().findAny();
```

May return any element.

✔ Faster

✅ 7. What if collector is not thread safe?

Wrong:

```
List<Integer> result = new ArrayList<>();

list.parallelStream()
   .forEach(result::add);
```

May cause:

- Data corruption
- Lost updates
- ArrayIndexOutOfBoundsException

Correct:

```
list.parallelStream()
   .collect(Collectors.toList());
```

Collectors are designed for safe parallel reduction.

✅ 8. What happens with duplicate key in toMap()?

```
List<String> list = Arrays.asList("A","B","A");

Map<String, Integer> map =
   list.stream()
      .collect(Collectors.toMap(
         s -> s,
         s -> s.length()
      ));
```

❌ Throws:
IllegalStateException: Duplicate key

Fix:

```
Collectors.toMap(
   s -> s,
   s -> s.length(),
   (oldValue, newValue) -> oldValue
);
```

✅ 9. What if sorted comparator inconsistent?

Bad comparator:

```
.sorted((a,b) -> 1)
```

May throw:

IllegalArgumentException: Comparison method violates its general contract!

Comparator must follow:

- Antisymmetric
- Transitive
- Consistent

✅ 10. What if stream source is modified?

```
List<Integer> list = new ArrayList<>(Arrays.asList(1,2,3));

list.stream()
   .forEach(n -> list.add(n));
```

❌ Throws:
ConcurrentModificationException

Because stream expects non-interference.

✅ 11. What if stream is consumed twice?

```
Stream<Integer> stream = list.stream();

stream.count();
stream.count();
```

❌ Throws:
IllegalStateException: stream has already been operated upon or closed

✔Streams are single-use

✅ 12. What if flatMap returns null?

Wrong:

```
list.stream()
   .flatMap(s -> null)
   .forEach(System.out::println);
```

❌ Throws:
NullPointerException

flatMap expects a Stream, not null.

Correct:

.flatMap(s -> Stream.empty())

✅ 13. What if infinite stream without limit?

```
Stream.iterate(1, n -> n + 1)
    .forEach(System.out::println);
```

Result:

Infinite loop → program never ends

Must use:

.limit(10)

✅ 14. What if order is removed?

```
list.parallelStream()
  .unordered()
  .limit(3)
  .forEach(System.out::println);
```

✔ Faster

✔ Order not guaranteed

Useful in large parallel data.

✅ 15. What if combiner is incorrect?

Example:

```
list.parallelStream()
  .reduce(0,
        (a,b) -> a + b,
        (a,b) -> a - b); // WRONG combiner
```

Output unpredictable.

Because:

- Accumulator = sum
- Combiner = subtraction
- Inconsistent logic

Combiner must match accumulator.

Correct:

(a,b) -> a + b

Senior-Level Summary

✔ Streams execute element-by-element
✔ Identity must be neutral
✔ Parallel streams require stateless lambdas
✔ Duplicate keys must have merge function
✔ flatMap cannot return null
✔ Infinite streams require short-circuit
✔ Combiner must be associative and consistent

If you want next:

✅ 30 FAANG-Level Output-Based Questions
✅ Deep Reduce + Collector Internal Mechanics
✅ Parallel Stream Edge Cases
✅ 50 Most Difficult Stream Problems

Total Questions: 140+

If you want, I can next provide:

- ✅ 50 most important questions with answers
- ✅ 25 real interview coding programs with full Java code
- ✅ Tricky stream output-based MCQs
- ✅ Internal working diagram explanation
- ✅ FAANG-level Stream questions
- ✅ Stream vs Reactive vs CompletableFuture comparison